

Podstawowe dane

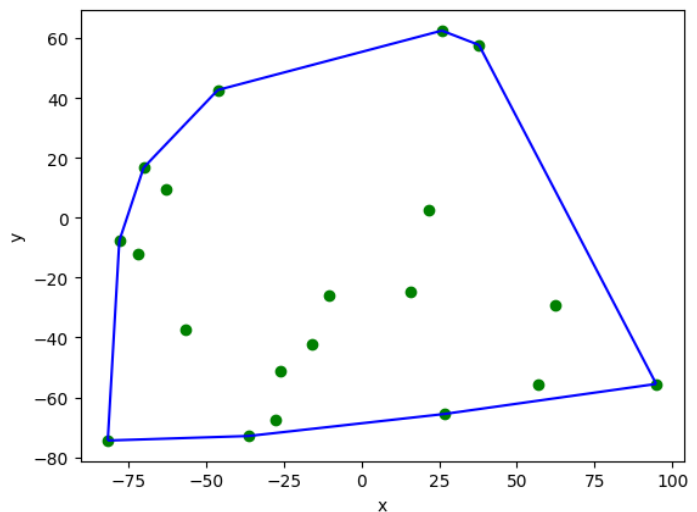
Imie i nazwisko	Robert Mesek
Data	01.11.2023
Tytuł	Algorytmy geometryczne
Numer ćwiczenia	2

Dane techniczne

Procesor	Intel i7-1165G7 x86_64
Język	Python 3.11.5
Biblioteki	NumPy 1.26.0, Pandas 2.1.1, Matplotlib 3.8.0
System operacyjny	WSL Ubuntu 22.04 (MS Windows 10 22H2)
Środowisko	conda 23.5.2

Czym jest otoczka wypukła?

Podzbiór płaszczyzny Q nazywamy wypukłym $\iff$, gdy dla dowolnej pary punktów $p, q \in Q$ odcinek $\overline{pq}$ jest całkowicie zawarty w Q . Otoczka wypukła $\mathcal{CH}(Q)$ zbioru Q jest najmniejszym wypukłym zbiorem zawierającym Q .



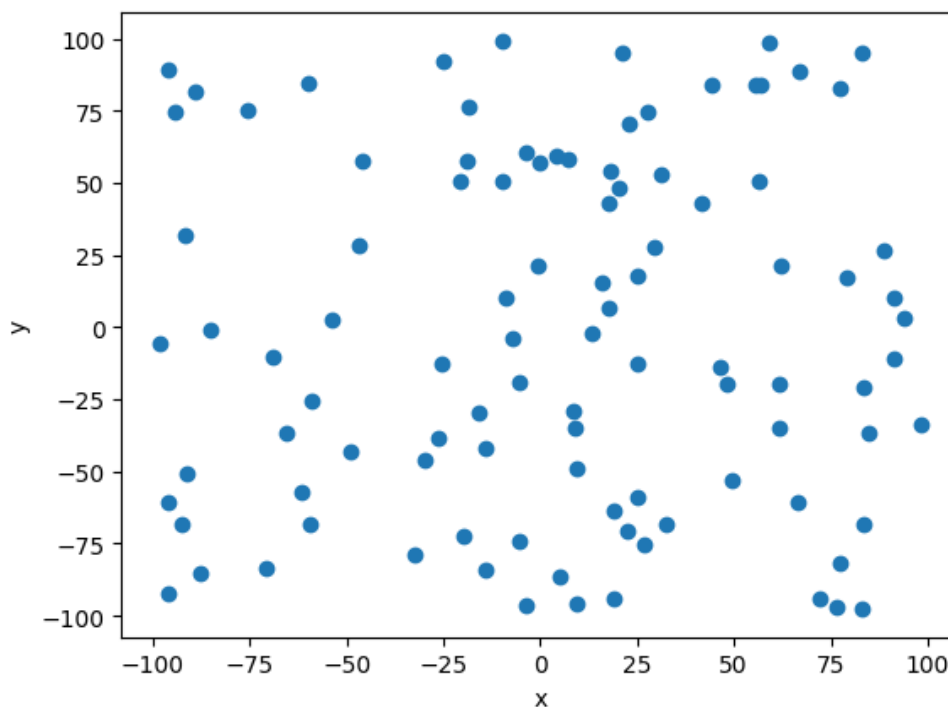
Zastosowania otoczku wypukłej

Otoczki wypukłe - w szczególności otoczki wypukłe w przestrzeni trójwymiarowej - są spotykane w różnych zastosowaniach. Na przykład używa się ich do przyspieszania wykrywania kolizji w animacji komputerowej. Przepuścimy, że chcemy sprawdzić, czy dwa obiekty $\mathcal{P}_\infty$ i $\mathcal{P}_\epsilon$ przecinają się. Jeśli przez większość czasu odpowiedź na to pytanie jest negatywna, to opłaca się następująca strategia. Przybliżamy obiekty przez prostsze obiekty $\widehat{\mathcal{P}}_\infty$ i $\widehat{\mathcal{P}}_\epsilon$, które zawierały oryginały. Jeśli chcemy sprawdzić, czy $\mathcal{P}_\infty$ i $\mathcal{P}_\epsilon$ przecinają się, najpierw sprawdzamy, czy przecinają się $\widehat{\mathcal{P}}_\infty$ i $\widehat{\mathcal{P}}_\epsilon$. Jeśli występuje ten przypadek, to powinniśmy wykonać test na oryginalnych obiektach, który jest przeprowadzanie znacznie kosztowniejszy. Sprawdzanie przecięcia otoczek wypukłych jest bardziej skomplikowane niż dla sfer - choć mimo to łatwiejsze niż dla obiektów niewypukłych - ale otoczki wypukłe mogą dużo lepiej przybliżać większość obiektów.

Zbiory punktów

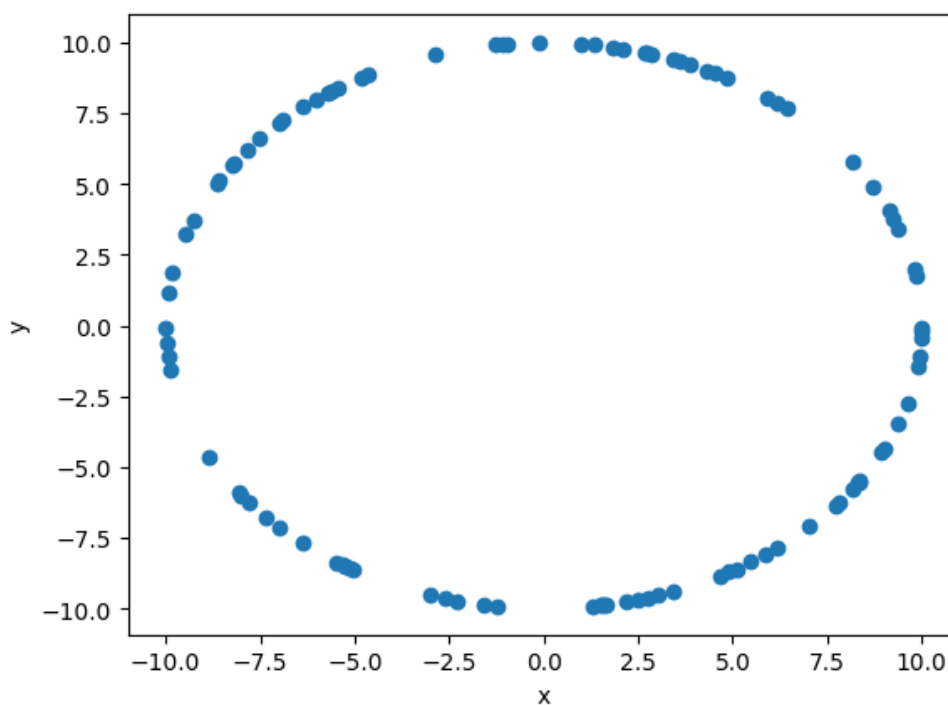
points_a

100 losowych punktów w przestrzeni 2D o współrzędnych z przedziału $x \in \langle -100, 100 \rangle$ oraz $y \in \langle -100, 100 \rangle$.



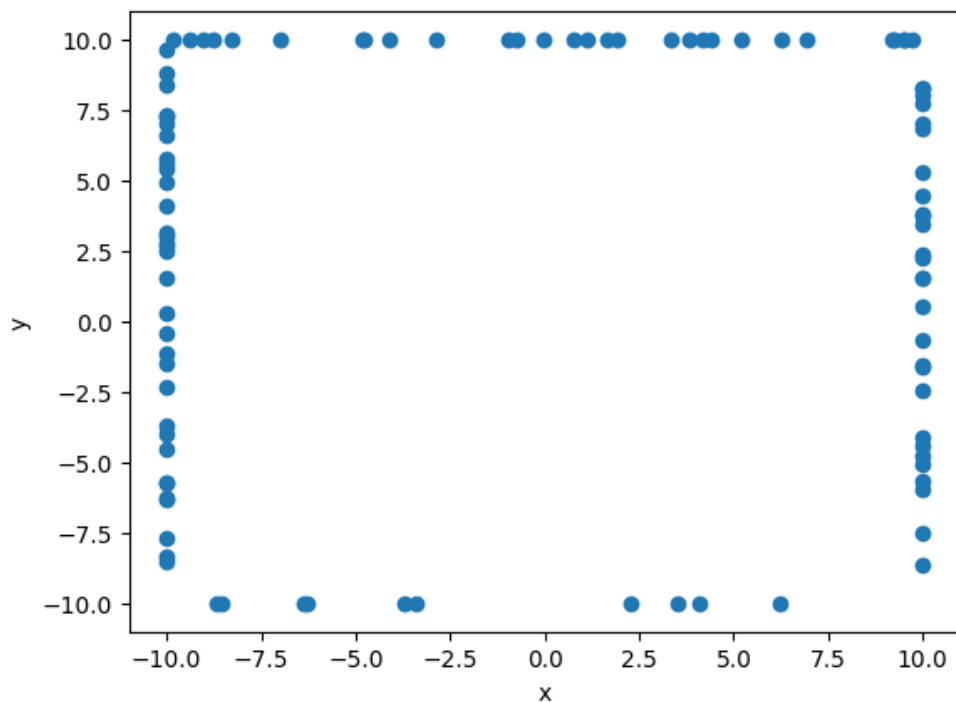
points_b

100 losowych punktów w przestrzeni 2D leżących na okręgu o środku $O = (0, 0)$ i promieniu $R = 10$.



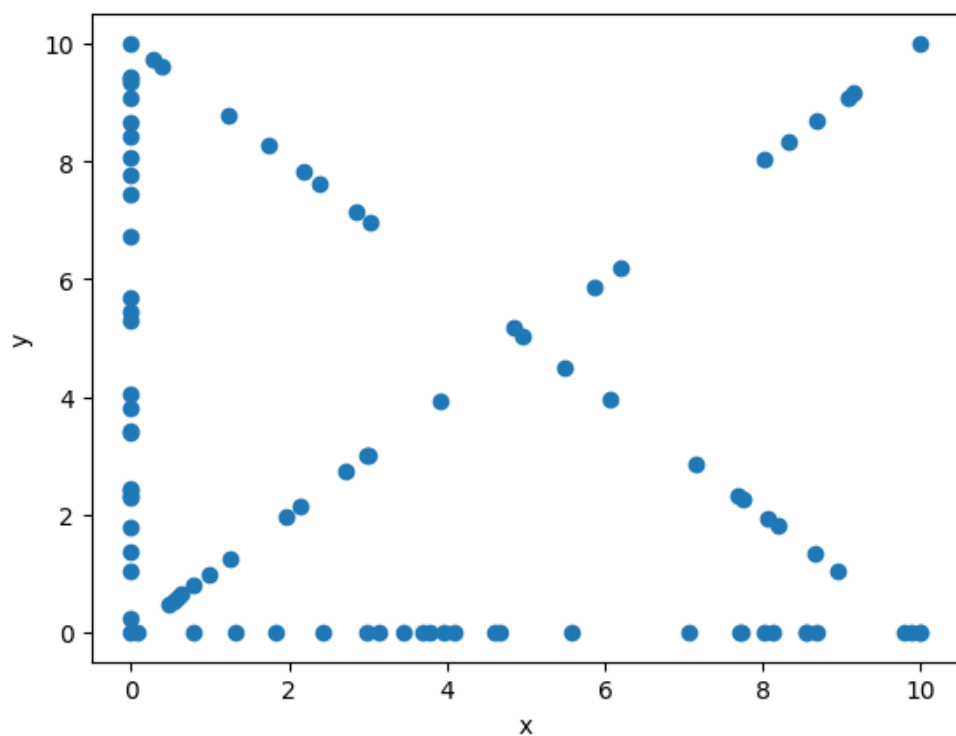
points_c

100 losowych punktów w przestrzeni 2D leżących na obwodzie prostokąta, którego wyznaczają wierzchołki $(-10, -10)$, $(10, -10)$, $(10, 10)$ oraz $(-10, 10)$.



points_d

po 25 punktów na dwóch bokach kwadratu leżących na osiach i po 20 punktów na przekątnych kwadratu, zawierający punkty wyznaczające kwadrat $(0, 0)$, $(10, 0)$, $(10, 10)$ oraz $(0, 10)$.



Algorytm Grahama oraz algorytm Jarvisa

Funkcje pomocnicze dla obu algorytmów

Reprezentuje punkt

```
class Point:
    def __init__(self, x, y, id=None):
        self.x = x
        self.y = y
        self.id = id # set to index

    def __repr__(self):
        return f"{self.id}. ({self.x}, {self.y})"

    def as_tuple(self):
        return (self.x, self.y)
```

Konwertuje listę krotek na listę punktów

```
def create_points(Q: list[tuple]):
    points = []
    for i, (x, y) in enumerate(Q):
        points.append(Point(x, y, i))
    return points
```

Konwertuje listę punktów na listę krotek

```
def to_list_of_tuples(points: list[Point]):
    list_of_tuples = []
    for point in points:
        list_of_tuples.append(point.as_tuple())
    return list_of_tuples
```

Sprawdza orientację punktu względem prostej korzystając z wyznacznika macierzy

```
def find_orient(a: Point, b: Point, c: Point):
    """
    Orientacja punktu "c" względem prostej "a b" obliczając wyznacznik macierzy 2x2
    :param a: pierwszy punkt tworzący naszą prostą
    :param b: drugi punkt tworzący naszą prostą
    :param c: punkt, którego położenie względem prostej chcemy znaleźć
    :return: wartość wyznacznika macierzy
    (> 0) => Counterclockwise
    (== 0) => Collinear
    (< 0) => Clockwise
    """
    return (a.x - c.x) * (b.y - c.y) - (a.y - c.y) * (b.x - c.x)
```

Oblicza odległość punktów

```
def distance(p1: Point, p2: Point):  
    return ((p2.x - p1.x) ** 2 + (p2.y - p1.y) ** 2) ** 0.5
```

Funkcje pomocnicze dla algorytmu grahama

Zwraca punkt startowy dla algorytmu

```
def find_start_point(points: list[Point]):  
    return min(points, key=lambda p: (p.y, p.x))
```

Sortuje punkty względem kąta jaki tworzą z prostą OX

```
def sort_points(p0: Point, points: list[Point], eps):  
    from functools import cmp_to_key  
  
    def compare(p1: Point, p2: Point, cmp_eps=eps):  
        """Will return a negative value for less-than, zero if the inputs are equal,  
        or a positive value for greater-than"""  
        orient = find_orient(p1, p2, p0)  
        if abs(orient) < cmp_eps:  
            if distance(p0, p1) > distance(p0, p2):  
                p2.id = None # None if collinear  
                return 1  
            else:  
                p1.id = None # None if collinear  
                return -1  
        elif orient > 0:  
            return -1  
        else:  
            return 1  
  
    points.sort(key=cmp_to_key(compare))
```

Usuwa punkty współliniowe wykorzystując flagę nadaną podczas sortowania

```
def remove_collinear(points: list[Point]):  
    for i in reversed(range(len(points))):  
        if points[i].id is None:  
            points.pop(i)
```

Algorytm Grahama

Algorytm Grahama tworzy otoczkę wypukłą poprzez utrzymywanie stosu S , w którym znajdują się punkty, które mogą, ale nie muszą tworzyć otoczki wypukłej. Za każdym razem jest wstawiany na stos (push) jeden punkt z zbioru punktów Q i jest on usuwany ze stosu (pop), jeżeli nie jest punktem $\mathcal{CH}(Q)$. Kiedy algorytm kończy się, stos S zawiera tylko punkty otoczki wypukłej $\mathcal{CH}(Q)$ w kierunku przeciwnym do ruchu wskazówek zegara.

Procedura **Graham – Build(Q)** przyjmuje zbiór punktów Q , gdzie $|Q| \geq 3$. Wywołuje ona funkcję **TOP(S)**, która zwraca punkt z góry stosu bez zmieniania S oraz **NEXT – TO – TOP(S)**, która zwraca punkt poniżej góry stosu S , bez zmieniania stosu. Funkcja **PUSH(p, S)** wstawia punkt p na stos S . Funkcja **POP(p, S)** usuwa punkt p ze stosu S .

Graham – Build(Q)

- 1) niech p_0 będzie punktem w zbiorze Q z najmniejszą współrzędną y , oraz najmniejszą współrzędną x w przypadku, gdy wiele punktów ma tą samą współrzędną x
- 2) niech $\langle p_1, p_2, \dots, p_m \rangle$ będzie pozostałym zbiorem punktów w Q posortowanym zgodnie z przeciwnym ruchem wskazówek zegara wokół punktu p_0 (jeżeli więcej niż jeden punkt ma ten sam kąt to usuwamy wszystkie punkty z wyjątkiem tego najbardziej oddalonego od p_0)
- 3) stwórz pusty stos S
- 4) **PUSH**(p_0, S)
- 5) **PUSH**(p_1, S)
- 6) **PUSH**(p_2, S)
- 7) **for** $i = 3$ **to** m
- 8) **while** kąt utworzony przez **NEXT – TO – TOP(S)**, **TOP(S)** oraz p_i tworzy lewostronny skręt
- 9) **POP**(S)
- 10) **PUSH**(p_i, S)
- 11) **return** S

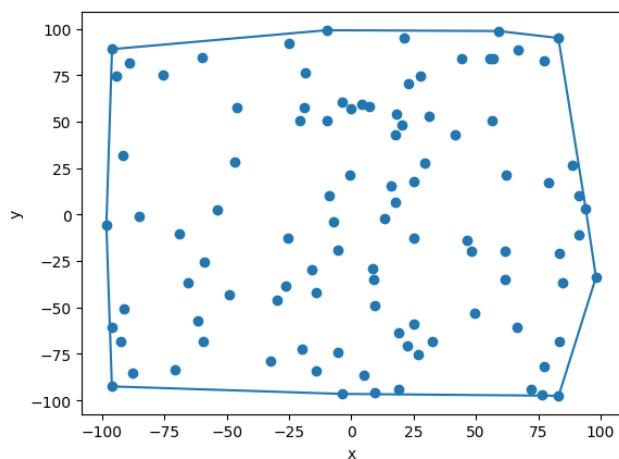
```
def graham_algorithm(Q, eps=10**-24):
    """
    Funkcja buduje otoczkę wypukłą dla podanego
    zbioru punktów Q algorymem Grahama
    :parm Q: zbiór punktów
    :return: tablica punktów w postaci krotek współrzędnych
    """
    points = create_points(Q)
    start_point = points.pop(find_start_point(points).id)
    sort_points(start_point, points, eps)
    remove_collinear(points) # according to angle (start_point -> a), (start_point -> b)
    stack = [start_point]
    for point in points:
        while len(stack) > 1 and find_orient(stack[-2], stack[-1], point) <= 0:
            stack.pop()
        stack.append(point)
    return to_list_of_tuples(stack)
```

Analiza wyników

Znaczenie kolorów

Niebieskie punkty - cały zbiór; Zielone punkty - rozpatrywane punkty przez algorytm, po usunięciu współliniowych; czerwona linia - aktualnie rozpatrywana krawędź; niebieska linia - znaleziona otoczka wypukła.

Zbiór A

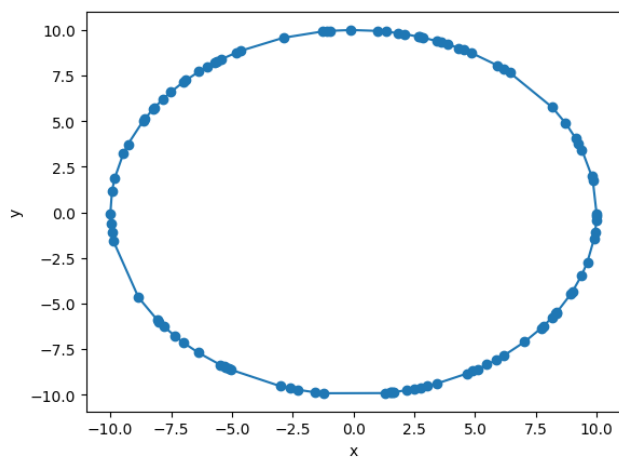


Rysunek Graham points_a

[images/graham-a.gif](#)

Animacja Graham points_a

Zbiór B

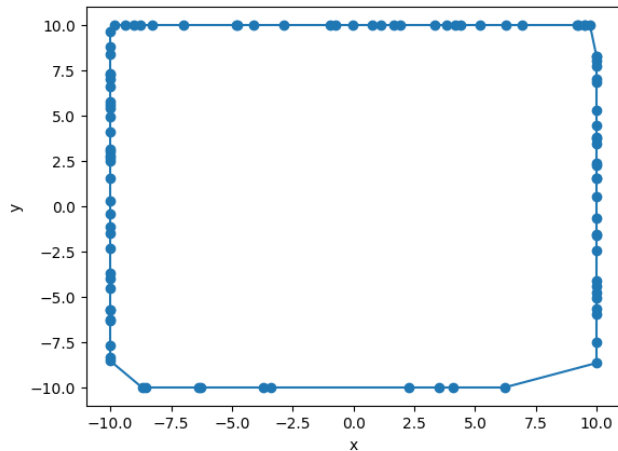


Rysunek Graham points_b

[images/graham-b.gif](#)

Animacja Graham points_b

Zbiór C

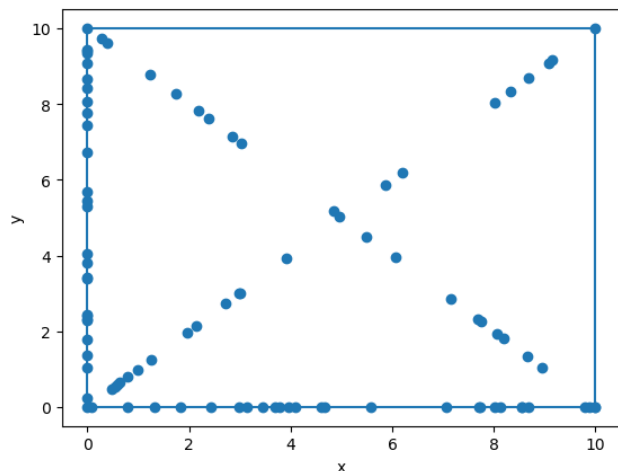


Rysunek Graham points_c

<images/graham-c.gif>

Animacja Graham points_c

Zbiór D



Rysunek Graham points_d

<images/graham-d.gif>

Animacja Graham points_d

Wnioski:

Analizując wyniki, widzimy że algorytm grahama dosyć dokładnie wyznaczył otoczki wypukłe dla wszystkich zbiorów.

Widzimy jednak, że usuwanie punktów współliniowych względem kąta jaki tworzą z osią OX, nie jest idealne. Dla zbioru C z powodu punktu punktu startowego, została odrzucona jedynie dolna krawędź, więc operacja odrzucania krawędzi nie zaoszczędziła dużo w tym przypadku. Zbiór D lepiej pokazuje, które punkty zostaną początkowo usunięte (bok ab oraz ad oraz proste wychodzące z punktu a).

Kolejnym problemem z jakim możemy się spotkać jest fakt, że decydując, który punkt zostanie wybrany porównujemy ich kąty, więc zmiana epsilon, nie wpływa w intuicyjny sposób na zwiększanie dokładności.

Funkcje pomocnicze dla algorytmu jarvisa

Zwraca punkt startowy dla algorytmu

```
def find_start_point(points: list[Point]):  
    return min(points, key=lambda p: (p.x, p.y))
```

Szuka kolejnego punktu kierując się kątem jaki tworzą z ostatnim punktem na otoczce, a w przypadku odcinków współliniowych wybiera najdalszy.

```
def find_next_point(start_point: Point, points: list[Point], eps):  
    end_point = points[0]  
    for point in points[1:]:  
        orient = find_orient(start_point, end_point, point)  
        if abs(orient) < eps and distance(start_point, point) > distance(start_point, end_point):  
            end_point = point  
        elif orient < 0:  
            end_point = point  
    return end_point
```

Algorytm Jarvisa

Algorytm Jarvisa oblicza otoczkę wypukłą dla zbioru punktów Q przez technikę zwaną owijaniem paczki (**package wrapping**) lub owijaniem prezentu (**gift wrapping**). Algorytm Jarvisa buduje sekwencję $H = \langle p_1, p_2, \dots, p_m \rangle$ będącą wierzchołkami $\mathcal{CH}(Q)$. Zaczynamy od punktu p_0 , następny punkt p_1 w otoczce wypukłej ma najmniejszy kąt w odniesieniu do p_0 (w przypadku takiego samego kąta - wybiera się punkt najdalej od p_0). Podobnie, gdy p_2 ma najmniejszy kąt w odniesieniu do p_1 , itd.. Zauważyć warto, że możemy tym sposobem obliczyć lewy i prawy łańcuch otoczki wypukłej $\mathcal{CH}(Q)$. Lewy łańcuch buduje się podobnie. Gdy osiągniemy najwyższy wierzchołek w prawym łańcuchu p_k , wybieramy wierzchołek p_{k+1} , który ma najmniejszy kąt w odniesieniu do p_k , ale od ujemnej osi- x . Można zaimplementować algorytm Jarvisa bez konstruowania pomocniczych łańcuchów - lewego i prawego. Taka implementacja utrzymuje śledzenie kąta ostatniej strony otoczki wypukłej i wymaga sekwencji kątów boków otoczki tylko rosnącej. (Patrząc na trzy ostatnie punkty jesteśmy w stanie obliczyć jaki punkt należy włączyć do $\mathcal{CH}(Q)$ w zależności od budowanego punktu)

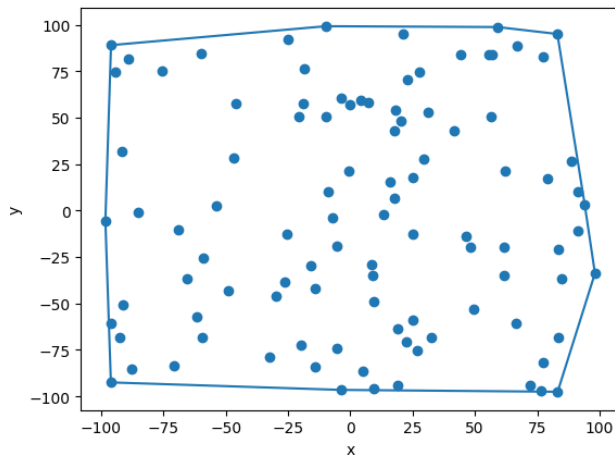
```
def jarvis_algorithm(Q, eps=10**-24):  
    """  
    Funkcja buduje otoczkę wypukłą dla podanego  
    zbioru punktów Q algorymem Jarvisa  
    :param Q: zbiór punktów  
    :return: tablica punktów w postaci krotek współrzędnych  
    """  
    points = create_points(Q)  
    hull = [find_start_point(points)]  
    for _ in range(len(points)):  
        next_point = find_next_point(hull[-1], points, eps)  
        if distance(hull[0], next_point) < eps:  
            break  
    hull.append(next_point)  
    return to_list_of_tuples(hull)
```

Analiza wyników

Znaczenie kolorów

Niebieskie punkty - cały zbiór punktów; czerwona linia - aktualnie rozpatrywana krawędź; niebieska linia - znaleziona otoczka wypukła.

Zbiór A

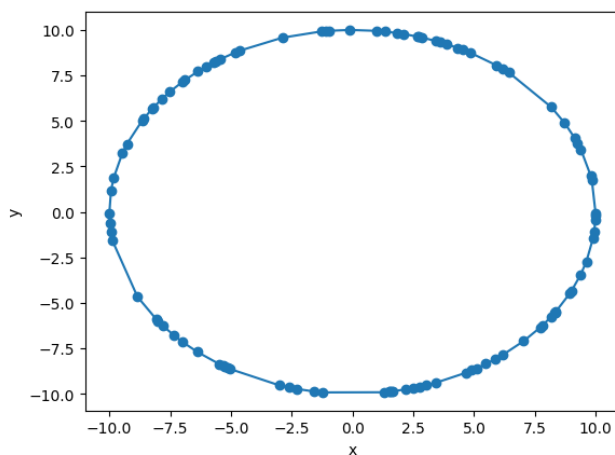


Rysunek Jarvis points_a

[images/jarvis-a.gif](#)

Animacja Jarvis points_a

Zbiór B

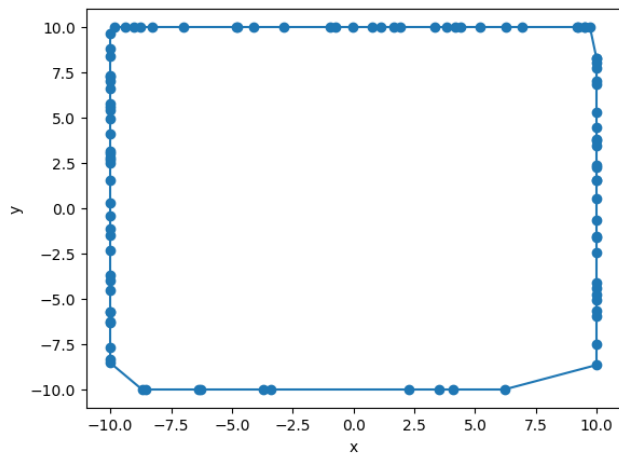


Rysunek Jarvis points_b

[images/jarvis-b.gif](#)

Animacja Jarvis points_b

Zbiór C

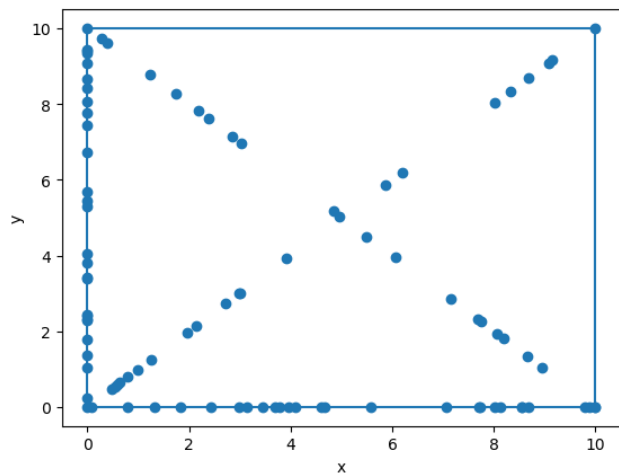


Rysunek Jarvis points_c

<images/jarvis-c.gif>

Animacja Jarvis points_c

Zbiór D



Rysunek Jarvis points_d

<images/jarvis-d.gif>

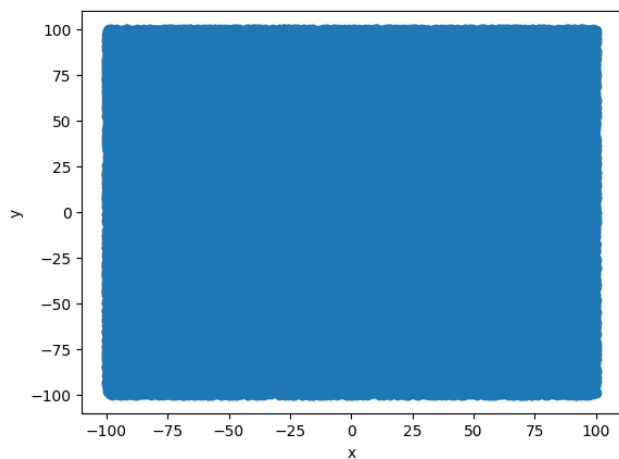
Animacja Jarvis points_d

Wnioski:

Analizując wyniki, widzimy że algorytm jarvisa wyznaczył dokładnie otoczki wypukłe dla wszystkich zbiorów.

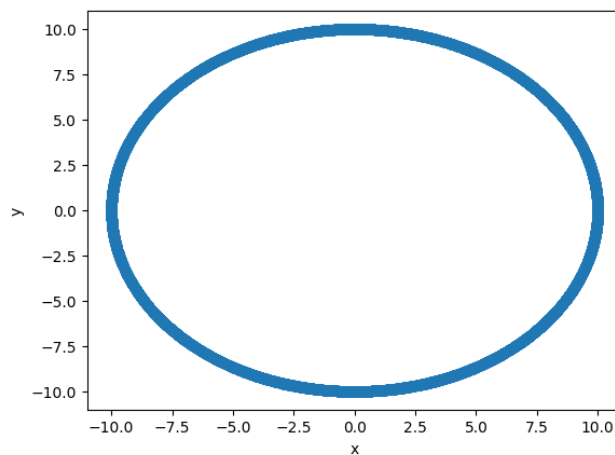
Wybór punktów jest realizowany w intuicyjny sposób.

Zbiory wykorzystane przy porównywaniu czasu działania



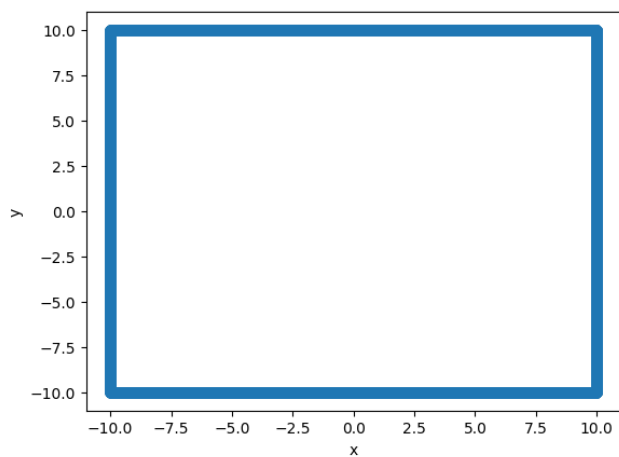
```
generate_uniform_points(-100, 100, 10**4)
```

Rysunek zbiór T.A



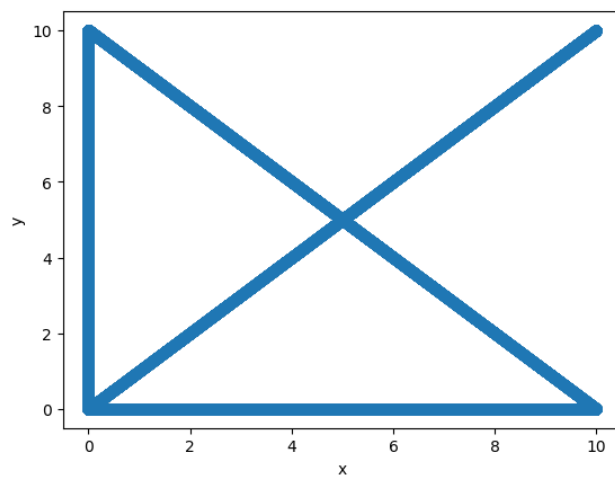
```
generate_circle_points((0, 0), 10, 10**4)
```

Rysunek zbiór T.B



```
generate_rectangle_points(  
(-10, -10), (10, -10), (10, 10), (-10, 10),  
10**6)
```

Rysunek zbiór T.C



```
generate_square_points(  
(0, 0), (10, 0), (10, 10), (0, 10),  
10**5, 10**5)
```

Rysunek zbiór T.D

Czas działania:

Algorytm	Zbiór T.A	Zbiór T.B	Zbiór T.C	Zbiór T.D
Graham	1.5 s	3.1 s	1.3 s	4.9 s
Jarvis	1.4 s	30.9 s	0.6 s	1.3 s

Tabela 1 Czasy działania

Wnioski

Wspólne dla obu algorytmów

- Oba służą do znajdowania otoczki wypukłej

Algorytm grahama

- Złożoność $O(n \log n)$ - (n to liczba punktów)
- Wyniki są wrażliwe na rozkład punktów, co spowodowane jest porównywaniem ich względem kąta który tworzą z osią OX (Widzimy to szczególnie w przypadku przekątnych kwadratu, gdzie jedna przekątna została potraktowana poprawnie jako punkty współliniowe, ale za to prawie cała druga przekątna była i tak sprawdzana).
- Jest zazwyczaj szybszy dla ogólnych danych, ale nie intuicyjnie na wyniki wpływa dokładność obliczeń, na co należy zwrócić uwagę.

Algorytm jarvisa

- Jego złożoność $O(n * k)$ - zależna od rozmiaru rozwiązania (k to ilość punktów otoczki)
- Idea działania algorytmu jest intuicyjna
- Jak pokazuje przykład przekątnych prostokąta może być szybszy od algorytmu grahama
- Zmiana epsilon lepiej odpowiada zmianie dokładności szukania punktów.

Bibliografia

- Wprowadzenie do algorytmów wydanie 3, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Stein Clifford